

# One-Way Functions and Lamport Signatures

- Signature from Trapdoor Permutation: RSA
- Signature from DLOG Permutation: ElGamal, Schnorr ...
- Signature from One-Way Function
  - **No Math!!!!!!**

Recall:  $f: X \rightarrow Y$  is a **one-way function** (OWF) if:

- easy: for all  $x \in X$  compute  $f(x)$
- inverting  $f$  is hard:

Example:  $f(x) = \text{AES}(x, o)$

Hardcore Predicate: A predicate  $p$  is hardcore predicate of OWF  $f$  if for any PPT. adversary,

$$\Pr_x[A(f(x)) = P(x)] \leq \frac{1}{2} + \text{negl}$$

Construction of Hardcore Predicate.

Recall:  $f: X \rightarrow Y$  is a **one-way function** (OWF) if:

- easy: for all  $x \in X$  compute  $f(x)$
- inverting  $f$  is hard:

Example:  $f(x) = \text{AES}(x, o)$

Signatures from OWF: Lamport-Merkle

- Signatures are long: stateless  $\Rightarrow > 40\text{KB}$  stateful  $\Rightarrow > 4\text{KB}$

# Lamport Signatures: One-Time Signature

- $sk = \begin{bmatrix} x_{00} & x_{01} & \dots & x_{0n-1} \\ x_{10} & x_{11} & \dots & x_{1n-1} \end{bmatrix}$
- $pk = \begin{bmatrix} y_{00} & y_{01} & \dots & y_{0n-1} \\ y_{10} & y_{11} & \dots & y_{1n-1} \end{bmatrix} = \begin{bmatrix} f(x_{00}) & f(x_{01}) & \dots & f(x_{0n-1}) \\ f(x_{10}) & f(x_{11}) & \dots & f(x_{1n-1}) \end{bmatrix}$
- $f$  is a one-way function

# Lamport Signatures: One-Time Signature

$$sk = \begin{bmatrix} x_{00} & \textcolor{red}{x}_{01} & \cdots & x_{0n-1} \\ \textcolor{red}{x}_{10} & x_{11} & \cdots & \textcolor{red}{x}_{1n-1} \end{bmatrix}$$

$$\text{Sign: } \sigma = (x_{0m_0}, x_{0m_1}, \cdots, x_{0m_{n-1}})$$

$$pk = \begin{bmatrix} y_{00} & \textcolor{red}{y}_{01} & \cdots & y_{0n-1} \\ \textcolor{red}{y}_{10} & y_{11} & \cdots & \textcolor{red}{y}_{1n-1} \end{bmatrix}$$

$$\text{Verify}(m_0 m_1 \cdots m_{n-1}, \sigma): f(\sigma_i) = y_{im_i}$$

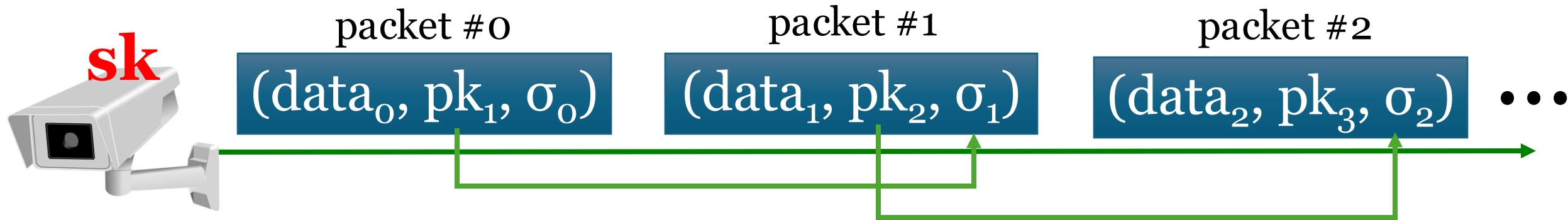
# Security Proof

- Not secure if being used to sign two messages
- Lamport's Signature Scheme is a secure one-time signature scheme (assuming  $f$  is a one-way function)
  - Proof of security

# Signing streams with fast one-time sigs

$(pk, sk)$ : key-pair for a many-time signature scheme

$(Gen_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)



Packet #0:  $(pk_1, sk_1) \leftarrow Gen_{1T}$  ,  $\sigma_0 \leftarrow S(sk, (data_0, pk_1))$

Packet #1:  $(pk_2, sk_2) \leftarrow Gen_{1T}$  ,  $\sigma_1 \leftarrow S_{1T}(sk_1, (data_1, pk_2))$

Packet #2:  $(pk_3, sk_3) \leftarrow Gen_{1T}$  ,  $\sigma_2 \leftarrow S_{1T}(sk_2, (data_2, pk_3))$

Recipient accepts **packet #2 = (data<sub>2</sub>, pk<sub>3</sub>, σ<sub>2</sub>)** once it verifies σ<sub>2</sub>

How does the recipient verify the signature σ<sub>2</sub> in packet #2?

Accept if σ<sub>0</sub> and σ<sub>1</sub> were valid and:

$$V_{1T}(pk_3, (data_2, pk_3), \sigma_2) = \text{“accept”}$$

$$V(pk, (data_2, pk_3), \sigma_2) = \text{“accept”}$$

$$V_{1T}(pk_2, (data_2, pk_3), \sigma_2) = \text{“accept”}$$

$$V(pk_2, (data_2, pk_3), \sigma_2) = \text{“accept”}$$

# From one-time to many-time

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

- **Gen:**

$$\text{Gen}_{1T} \longrightarrow \boxed{(\text{pk}_{0123}, \text{sk}_{0123})}$$

$(\text{pk}_{01}, \text{sk}_{01})$

$(\text{pk}_{23}, \text{sk}_{23})$

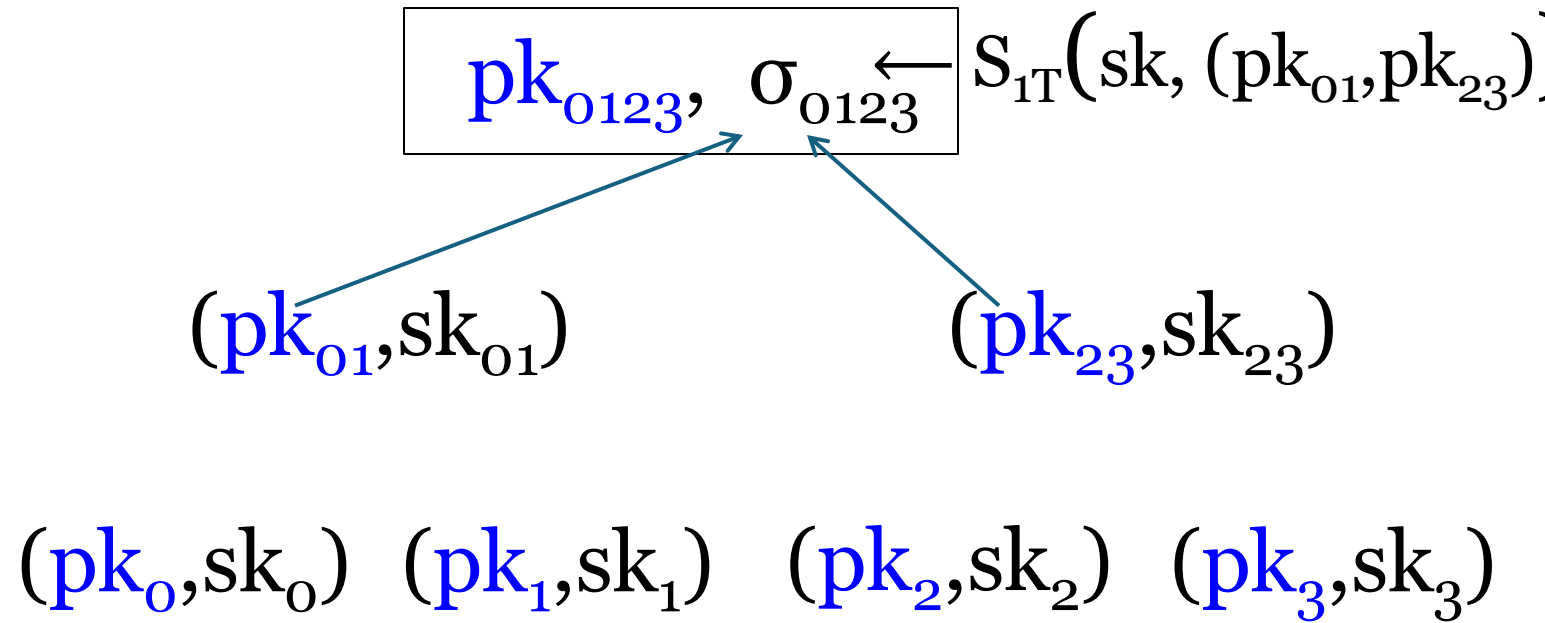
$(\text{pk}_0, \text{sk}_0)$   $(\text{pk}_1, \text{sk}_1)$   $(\text{pk}_2, \text{sk}_2)$   $(\text{pk}_3, \text{sk}_3)$

# Construction

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

- **Gen:**

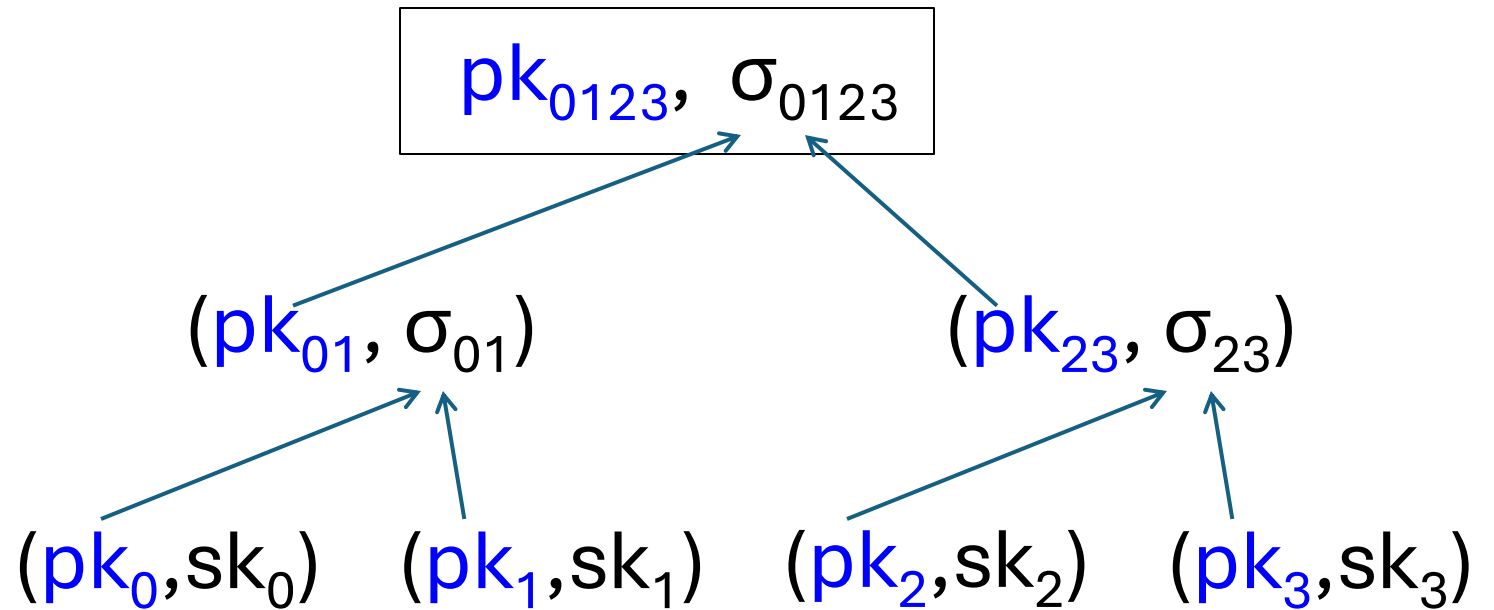


# Construction

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

- **Gen:**



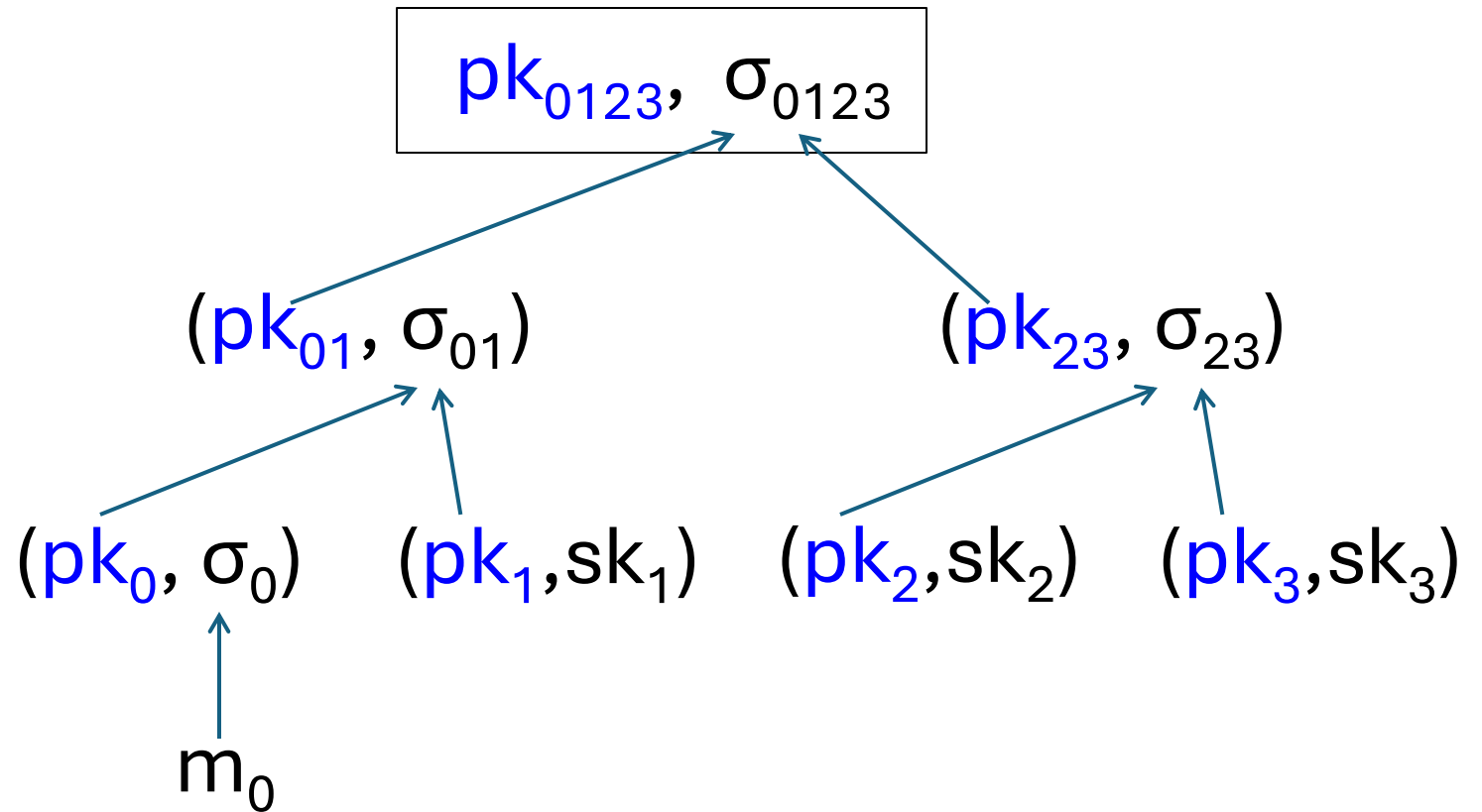
# Construction

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

Sig. on msg  $m_0$ :

$(\sigma_{0123}, \sigma_{01}, \mathbf{\sigma}_0,$   
 $pk_{01}, pk_{23}, pk_0, pk_1)$



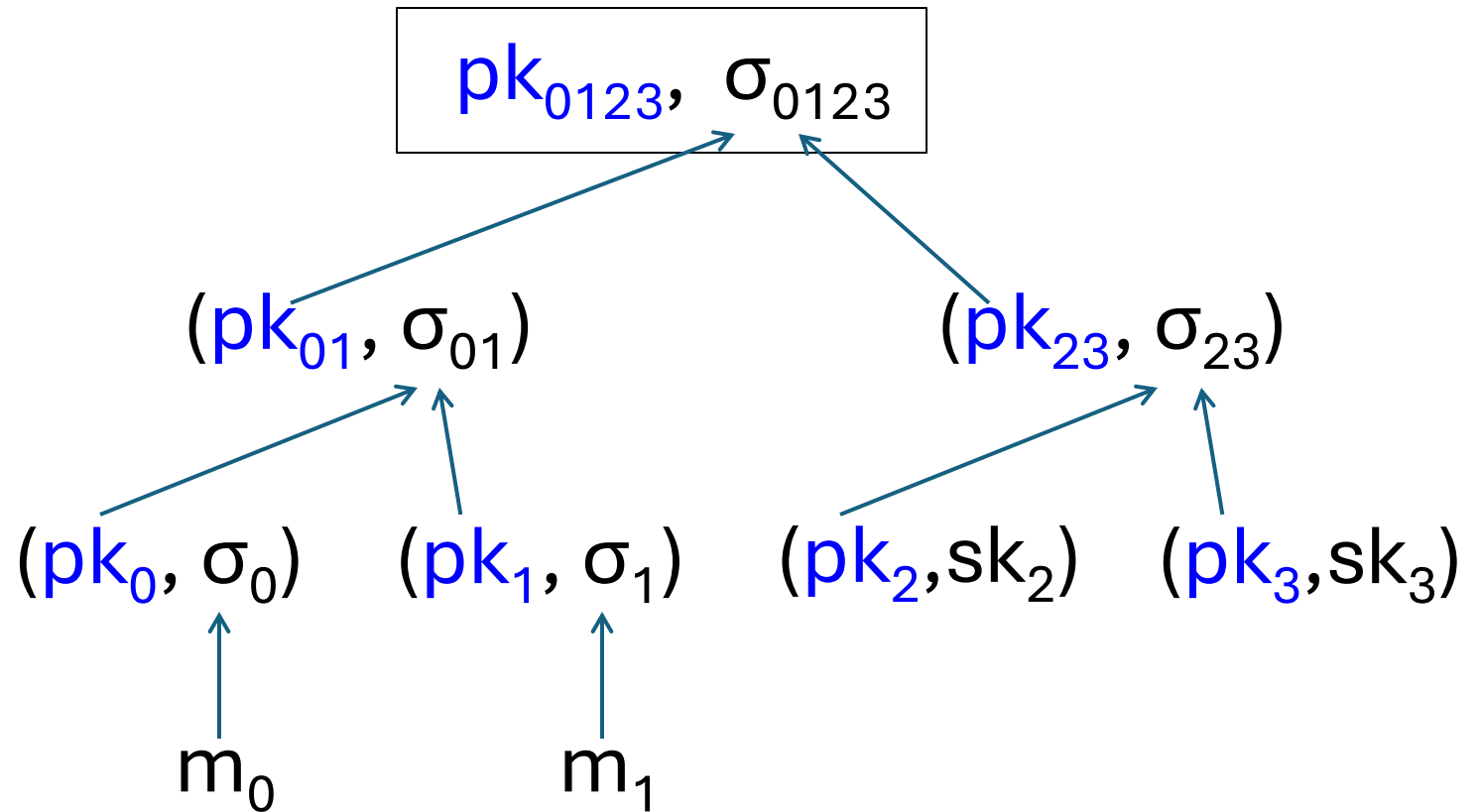
# Construction

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

Sig. on msg  $m_1$ :

$(\sigma_{0123}, \sigma_{01}, \mathbf{\sigma}_1,$   
 $pk_{01}, pk_{23}, pk_0, pk_1)$



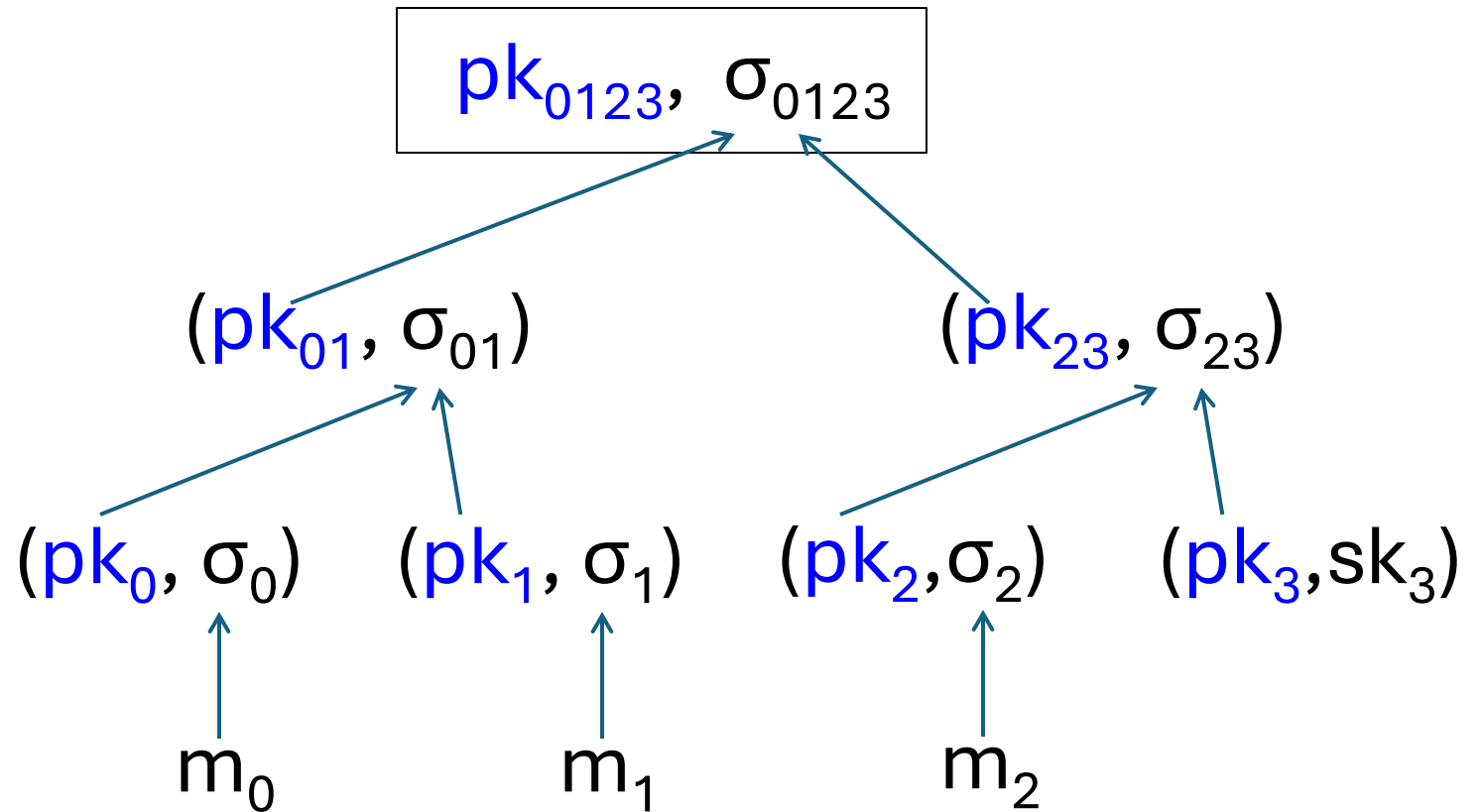
# Construction

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

Sig. on msg  $m_2$ :

$(\sigma_{0123}, \sigma_{23}, \mathbf{\sigma}_2,$   
 $\text{pk}_{01}, \text{pk}_{23}, \mathbf{pk}_2, \mathbf{pk}_3)$



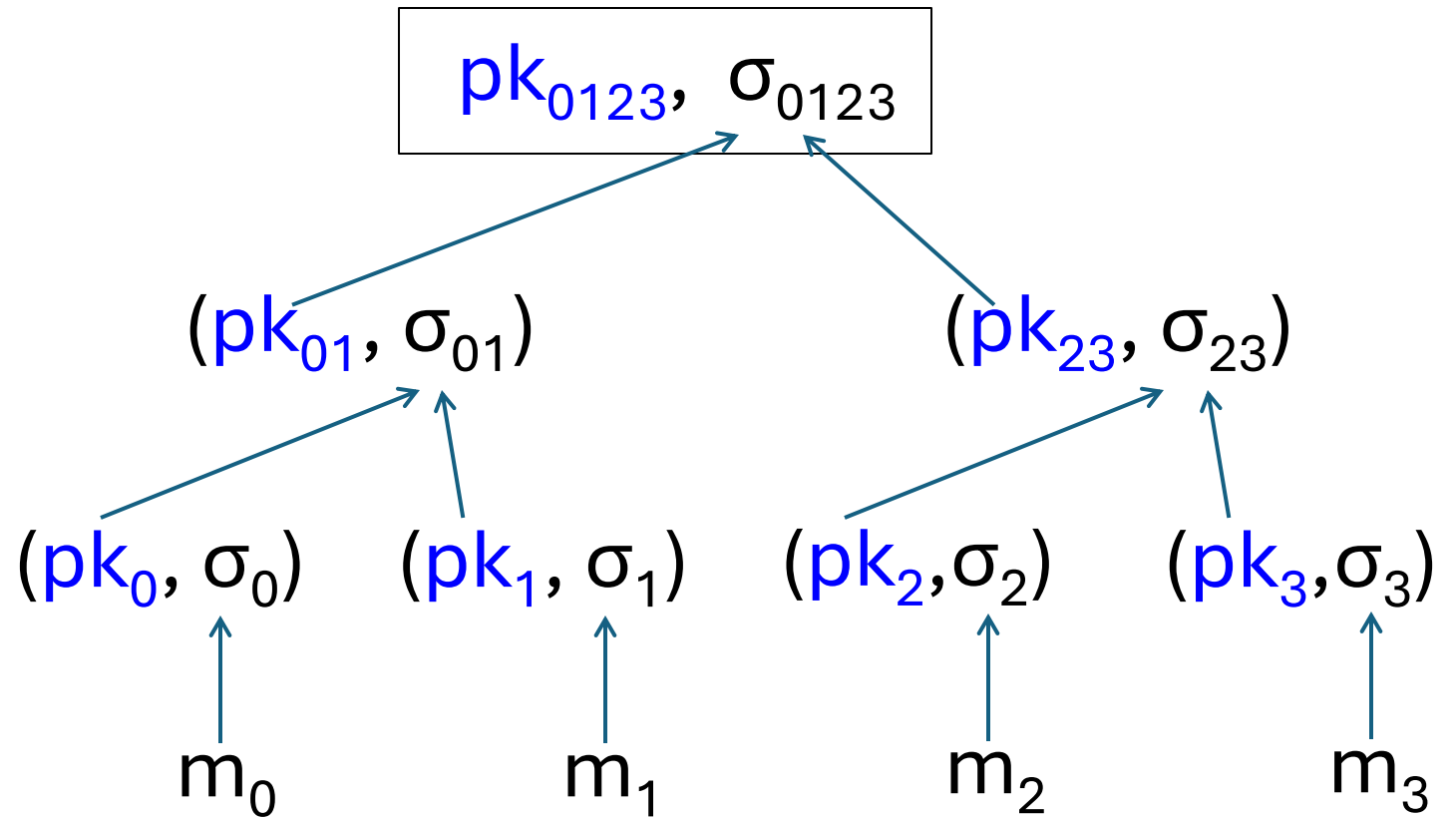
# Construction

$(\text{Gen}_{1T}, S_{1T}, V_{1T})$ : secure one-time signature (fast)

Four-time signature: (stateful version)

Sig. on msg  $m_3$ :

$(\sigma_{0123}, \sigma_{23}, \mathbf{\sigma}_3,$   
 $\text{pk}_{01}, \text{pk}_{23}, \mathbf{pk}_2, \mathbf{pk}_3)$



# More generally: $2^d$ -time signature

Tree of depth  $d$ :

- Every signature contains  $d+1$  one-time signatures along with associated  $pk$ 's

Tree is generated on-the fly:

- Signer stores only  $d$  secret keys at a time

Stateful signature:

- Signer maintains a counter indicating which leaf to use for signature
- Every leaf must only be used once!

